SymPy Bug Report

1 Bug 36: Wrong-Sign Antiderivative on the Negative Real Branch

1.1 Minimal Reproducer

```
from sympy import N, diff, integrate, sqrt, symbols

x = symbols("x")
integrand = 1/(x*sqrt(x**2 + 1))
F = integrate(integrand, x)
dF = diff(F, x)

print("antiderivative:", F)
print("derivative:", dF)
print("integrand at x=-2:", N(integrand.subs(x, -2), 50))
print("derivative at x=-2:", N(dF.subs(x, -2), 50))
print("difference at x=-2:", N((dF - integrand).subs(x, -2), 50))
```

1.2 Observed Behavior

```
antiderivative: -asinh(1/x)
derivative: 1/(x**2*sqrt(1 + x**(-2)))
integrand at x=-2: -0.22360679774997896964091736687312762354406183596115
derivative at x=-2: 0.22360679774997896964091736687312762354406183596115
difference at x=-2: 0.44721359549995793928183473374625524708812367192231
```

SymPy returns $-\operatorname{asinh}(1/x)$. At the regular real point $x = -2$, the derivative of this returned expression is positive, while the original integrand is negative. The mismatch is not a formatting issue: differentiating an indefinite integral should recover the integrand locally on the branch where the returned expression is being used.

1.3 Expected Behavior

For

$$f(x) = \frac{1}{x\sqrt{x^2+1}},$$

SymPy should return an antiderivative whose derivative equals $f(x)$ on the negative real interval as well as on the positive real interval, or it should return a result carrying the necessary branch condition. A single unconditional expression whose derivative has the opposite sign for $x < 0$ is mathematically incorrect.

1.4 Mathematical Explanation

SymPy's returned expression is

$$F(x) = -\operatorname{asinh}\left(\frac{1}{x}\right).$$

Differentiating gives

$$F'(x) = \frac{1}{x^2\sqrt{1+x^{-2}}}.$$

For real $x \neq 0$,

$$\sqrt{1+x^{-2}} = \sqrt{\frac{x^2+1}{x^2}} = \frac{\sqrt{x^2+1}}{|x|}.$$

Therefore

$$F'(x) = \frac{|x|}{x^2\sqrt{x^2+1}} = \frac{1}{|x|\sqrt{x^2+1}}.$$

On the positive real branch this agrees with $1/(x\sqrt{x^2+1})$. On the negative real branch it is the negative of the requested integrand:

$$\frac{1}{|x|\sqrt{x^2+1}} = -\frac{1}{x\sqrt{x^2+1}} \qquad (x<0).$$

At $x=-2$, this gives $F'(-2) = 1/(2\sqrt{5})$, while the integrand is $-1/(2\sqrt{5})$. The returned expression is thus a one-branch antiderivative, not an equivalent global antiderivative up to an additive constant.

1.5 Source-Level Diagnosis

The diagnosis narrows the failure to the indefinite Meijer integration path in

`sympy/integrals/meijerint.py`,

specifically `_meijerint_indefinite_1` around lines 1704–1740, with the closest location identified near line 1736. The default `integrate` path in `sympy/integrals/integrals.py` tries other methods first; for this integrand, the artifact reports that `heurisch`, manual integration, and Risch do not produce the final answer, and `integrate(..., meijerg=False)` remains unevaluated. The result comes from `meijerint_indefinite`.

The traced call path is `integrate` to `meijerint_indefinite` to `_meijerint_indefinite_1` to `_rewrite1`. For the reproducer, `_rewrite1` represents the integrand as an x^{-1} power factor times a Meijer G-function of x^2:

$$(1,\ 1/x,\ [(1/\sqrt{\pi},\ 0,\ \text{meijerg}(((1/2),()),((0),()),x^2))],\ \text{True}).$$

The problematic behavior appears in the branch-blind substitution and reconstruction step:

```
a, b = _get_coeff_exp(g.argument, x)
_, c = _get_coeff_exp(po, x)
...
fac_ = fac * C * x**(1 + c) / b
rho = (c + 1)/b
...
r = hyperexpand(r.subs(t, a*x**b), place=place)
res += powdenest(fac_*r, polar=True)
```

For this input, the substitution is effectively $t = x^2$ while the prefactor contains $1/x$. That substitution identifies the positive and negative real branches of x. The resulting Meijer antiderivative is then hyperexpanded to $-\operatorname{asinh}(1/x)$ without a condition recording that the expression differentiates correctly only on one real branch. This explains the observed mathematical failure: the square-root simplification hidden in the derivative uses $|x|$, while the original integrand uses x.

The suggested fix direction in the diagnosis is to avoid branch-losing substitutions such as $x \mapsto x^2$ in the indefinite Meijer path when odd powers such as $1/x$ are present, unless the method can return a piecewise or otherwise branch-conditioned antiderivative. At minimum, the Meijer result should be checked against the original integrand under real branch assumptions, or declined when the derivative check is branch-dependent.

1.6 Additional Failing Instances

The same sign error occurs for the family

$$\int \frac{dx}{x\sqrt{x^2+a}}, \qquad a \in \{1, 2, 3, 4, 5, 9\}.$$

For positive a, SymPy returns the corresponding one-branch expression

$$F_a(x) = -\frac{\operatorname{asinh}(\sqrt{a}/x)}{\sqrt{a}},$$

with the obvious simplifications when a is a square. Differentiation gives the positive value $1/(2\sqrt{4+a})$ at $x = -2$, while the original integrand has value $-1/(2\sqrt{4+a})$. The expected behavior is the same throughout the family because $\sqrt{x^2+a}/x$ and $\sqrt{1+a/x^2}$ differ by the sign of x on the real line.

Input / Expression	**SymPy behavior**	**Expected behavior**
$a = 1:\ 1/(x\sqrt{x^2+1})$	$F'(-2) = 1/(2\sqrt{5})$	$-1/(2\sqrt{5})$
$a = 2:\ 1/(x\sqrt{x^2+2})$	$F'(-2) = 1/(2\sqrt{6})$	$-1/(2\sqrt{6})$
$a = 3:\ 1/(x\sqrt{x^2+3})$	$F'(-2) = 1/(2\sqrt{7})$	$-1/(2\sqrt{7})$
$a = 4:\ 1/(x\sqrt{x^2+4})$	$F'(-2) = 1/(4\sqrt{2})$	$-1/(4\sqrt{2})$
$a = 5:\ 1/(x\sqrt{x^2+5})$	$F'(-2) = 1/6$	$-1/6$
$a = 9:\ 1/(x\sqrt{x^2+9})$	$F'(-2) = 1/(2\sqrt{13})$	$-1/(2\sqrt{13})$

1.7 Related Issues Assessment

The best-effort deduplication check found documentation and background material related to the general family of branch-sensitive substitutions, including SymPy documentation warning that quadratic substitutions are acceptable only when the transformed integrand does not depend on the sign of the solutions. It did not identify a public SymPy issue, pull request, Stack Overflow report, mailing-list post, or release note for this exact integrand, the output $-\operatorname{asinh}(1/x)$, or the derivative sign mismatch at a negative real point. The deduplication verdict is therefore a new instance of a known failure mode: the broader branch-loss mechanism is understood, but this exact reproducible indefinite-integration case appears previously unreported and remains individually actionable as a focused issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import N, diff, integrate, sqrt, symbols


@pytest.mark.parametrize("a", [1, 2, 3, 4, 5, 9])
def test_integrate_x_sqrt_x2_plus_a_negative_branch_sign(a):
    x = symbols("x")
    integrand = 1/(x*sqrt(x**2 + a))
    F = integrate(integrand, x)
    difference = N((diff(F, x) - integrand).subs(x, -2), 50)

    assert abs(complex(difference)) < 1e-40
```